

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

UNIT II CLASSES OBJECTS AND CONSTRUCTORS

Object Oriented Programming concepts: Encapsulation, Abstraction, Defining classes and objects in java, Constructors: Default, Parameterized, Constructor overloading, Static variables and methods, Access specifiers: Public, Private, protected, default, Method Overloading, Applications: Student Record system, Basic user registration models,

2.1 OBJECT ORIENTED CONCEPTS

OOPs (Object-Oriented Programming System) / Object Oriented Thinking, Object means a real-world entity such as a pen, chair, table, computer, watch, etc. Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects.

It simplifies software development and maintenance by providing some concepts:

- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Apart from these concepts, there are some other terms which are used in Object-Oriented design:

- Coupling
- Cohesion
- Association
- Aggregation
- Composition

Object

- Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.
- An Object can be defined as an instance of a class.
- An object contains an address and takes up some space in memory.

- ❑ Objects can communicate without knowing the details of each other's data or code.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

- ❑ The only necessary thing is the type of message accepted and the type of response returned by the objects.

Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

Class

- ❑ Collection of objects is called class. It is a logical entity.
- ❑ A class can also be defined as a blueprint from which you can create an individual object.
- ❑ Class doesn't consume any space.

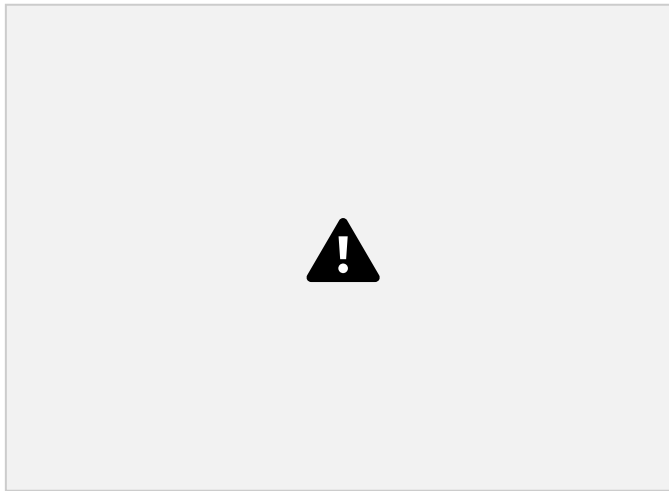
Inheritance

- ❑ When one object acquires all the properties and behaviors of a parent object, it is known as inheritance.
- ❑ It provides code reusability.
- ❑ It is used to achieve runtime polymorphism.

Polymorphism

- ❑ If one task is performed in different ways, it is known as polymorphism.
- ❑ For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.
- ❑ In Java, we use method overloading and method overriding to achieve polymorphism.
- ❑ Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

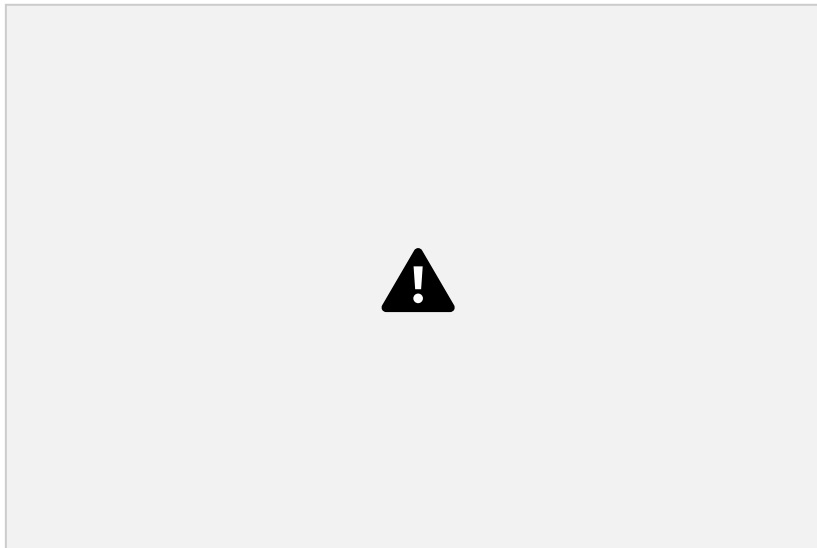
UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Abstraction

☐ Hiding internal details and showing functionality is known as abstraction. ☐ For example phone call, we don't know the internal processing.

☐ In Java, we use abstract class and interface to achieve abstraction.



Encapsulation

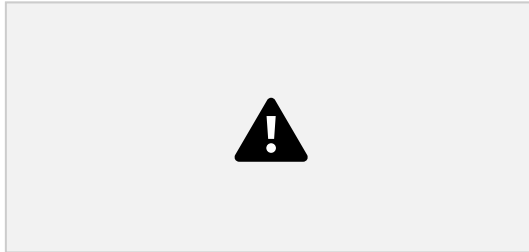
☐ Binding (or wrapping) code and data together into a single unit are known as encapsulation. ☐ For example, a capsule, it is wrapped with different medicines.

☐ A java class is the example of encapsulation.

☐ Java bean is the fully encapsulated class because all the data members are private here.

☐ Encapsulation is the process of combining data and code into a single unit (object / class). In OOP, every object is associated with its data and code.

☐ In programming, data is defined as variables and code is defined as methods. The java programming language uses the class concept to implement encapsulation.



Coupling

Coupling refers to the knowledge or information or dependency of another class. It arises when classes are aware of each other. If a class has the details information of another class, there is strong coupling. In Java, we use private, protected, and public modifiers to display the visibility level of a class, method, and field. You can use interfaces for the weaker coupling because there is no concrete implementation.

Cohesion

Cohesion refers to the level of a component which performs a single well-defined task. A single well-defined task is done by a highly cohesive method. The weakly cohesive method will split the task into separate parts.

The java.io package is a highly cohesive package because it has I/O related classes and interface. However, the java.util package is a weakly cohesive package because it has unrelated classes and interfaces

Association

Association represents the relationship between the objects. Here, one object can be associated with one object or many objects. There can be four types of association between the objects:

- o One to One

- o One to Many

- o Many to One, and

- o Many to Many

Let's understand the relationship with real-time examples. For example, One country can have one prime minister (one to one), and a prime minister can have many ministers (one to many). Also, many MP's can have one prime minister (many to one), and many ministers can have many departments (many to many).

Association can be unidirectional or bidirectional.

Aggregation

Aggregation is a way to achieve Association. Aggregation represents the relationship where one object contains other objects as a part of its state. It represents the weak relationship between objects. It is also termed as a hasa relationship in Java. Like, inheritance represents the is-a relationship. It is another way to reuse objects.

Composition

The composition is also a way to achieve Association. The composition represents the relationship where one object contains other objects as a part of its state. There is a strong relationship between the containing object and the dependent object. It is the state where containing objects do not have an independent existence. If you delete the parent object, all the child objects will be deleted automatically.

A way of viewing world

A way of viewing the world is an idea to illustrate the object-oriented programming concept with an example of a real-world situation.

Let us consider a situation, I am at my office and I wish to get food to my family members who are at my home from a hotel. Because of the distance from my office to home, there is no possibility of getting food from a hotel myself. So, how do we solve the issue?

To solve the problem, let me call zomato (an agent in food delivery community), tell them the variety and quantity of food and the hotel name from which I wish to deliver the food to my family members. Look at the following image.



2.2 DEFINING OBJECTS AND CLASSES IN JAVA

Java Classes

☐ Java is an object-oriented programming language, so everything in java program must be based on the object concept. In a java programming language, the class concept defines the skeleton of an object.

☐ The java class is a template of an object. The class defines the blueprint of an object. Every class in java forms a new data type.

☐ Once a class got created, we can generate as many objects as we want. Every class defines the properties and behaviors of an object. All the objects of a class have the same properties and behaviors that were defined in the class.

☐ Every class of java programming language has the following characteristics.

- o **Identity** - It is the name given to the class.

- o **State** - Represents data values that are associated with an object.

- o **Behavior** - Represents actions can be performed by an object

Look at the following picture to understand the class and object concept.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Creating a Class

In java, we use the keyword class to create a class. A class in java contains properties as variables and behaviors as methods. Following is the syntax of class in the java.

Syntax

```
class <ClassName>{  
  
    data members declaration;  
  
    methods defination;  
  
}
```

- The ClassName must begin with an alphabet, and the Upper-case letter is preferred.
- The ClassName must follow all naming rules.

Creating an Object

In java, an object is an instance of a class. When an object of a class is created, the class is said to be instantiated. All the objects that are created using a single class have the same properties and methods. But the value of properties is different for every object. Following is the syntax of class in the java.

Syntax

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
<ClassName> <objectName> = new <ClassName>( );
```

- The objectName must begin with an alphabet, and a Lower-case letter is preferred. •

The objectName must follow all naming rules.

Java Methods

A method is a block of statements under a name that gets executed only when it is called. Every method is used to perform a specific task. The major advantage of methods is code reusability (define the code once, and use it many times).

In a java programming language, a method is defined as a behavior of an object. That means, every method in java must belong to a class.

- Every method in java must be declared inside a class.
- Every method declaration has the following characteristics.

☐ returnType - Specifies the data type of a return value.

☐ name - Specifies a unique name to identify it.

☐ parameters - The data values it may accept or receive.

☐ { } - Defines the block belongs to the method.

Creating a method

A method is created inside the class and it may be created with any access specifier. However, specifying access specifier is optional.

Following is the syntax for creating methods in java.

Syntax

```
class <ClassName>{
    <accessSpecifier> <returnType> <methodName>( parameters ){
    ...
    block of statements;
    ...
}
```


}

}

- The methodName must begin with an alphabet, and the Lower-case letter is preferred.
- The methodName must follow all naming rules.
- If you don't want to pass parameters, we ignore it.
- If a method defined with return type other than void, it must contain the return statement, otherwise, it may be ignored.

Calling a method

In java, a method call precedes with the object name of the class to which it belongs and a dot operator. It may call directly if the method defined with the static modifier. Every method call must be made, as to the method name with parentheses (), and it must terminate with a semicolon.

Syntax

<objectName>.<methodName>(actualArguments);

- The method call must pass the values to parameters if it has.
- If the method has a return type, we must provide the receiver.

Let's look at the following example java code.

Example

```
import java.util.Scanner;

public class JavaMethodsExample
{
    int sNo;

    String name;

    Scanner read = new Scanner(System.in);

    void readData()
    {
```

```

System.out.print("Enter Serial Number: ");

sNo = read.nextInt();

System.out.print("Enter the Name: ");

name = read.next();

}

static void showData(int sNo, String name) {

System.out.println("Hello, " + name + "! your serial number is " + sNo);

}

public static void main(String[] args)

{

JavaMethodsExample obj = new JavaMethodsExample();

obj.readData(); // method call using object

showData(obj.sNo, obj.name); // method call without using object

}

}

```

2.3 CONSTRUCTORS: DEFAULT, PARAMETERIZED

A constructor is a special method of a class that has the same name as the class name. The constructor gets executed automatically on object creation. It does not require the explicit method call. A constructor may have parameters and access specifiers too. In java, if you do not provide any constructor the compiler automatically creates a default constructor. They look like methods but have some key rules:

Key Points About Constructors

1. Same name as the class – No return type, not even `void`.
2. Automatically called when an object is created using `new`.
3. Used to set initial values for object attributes or run setup code.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

4. If you don't define a constructor, Java automatically provides a ****default constructor**** (no parameters, does nothing).

Types of Constructors

1. Default Constructor – No parameters.
 2. Parameterized Constructor – Accepts arguments to set initial values.
 3. Copy Constructor (not built-in like C++, but can be defined manually to copy object data).
- 1) **Default Constructor**

Provided automatically by java if you don't write any constructor.

Sets default values (eg 0 for numbers, null for objects)

Example

```
class Car {  
  
    String brand;  
  
    int year;  
  
    // Default constructor  
  
    Car() {  
  
        brand = "Unknown";  
  
        year = 2000;  
  
        System.out.println("Car object created!");  
  
    }  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car c1 = new Car(); // Constructor called automatically
```

```
System.out.println(c1.brand + " " + c1.year);
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
}
```

```
}
```

2) Parameterized Constructor

Accepts arguments(Parameters) to initialize object values.

Example

```
class Car {  
  
    String brand;  
  
    int year;  
  
    // Parameterized constructor  
  
    Car(String b, int y) {  
  
        brand = b;  
  
        year = y;  
  
    }  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car c1 = new Car("Toyota", 2022);  
  
        System.out.println(c1.brand + " " + c1.year);  
  
    }  
  
}
```

3) **Copy constructor**

In Java, a copy constructor is not built-in like in C++, but you can create one manually to make a

copy of an existing object. It's just a constructor that takes another object of the same class as a parameter and copies its data.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

Example of Copy Constructor in Java

```
class Car {  
  
    String brand;  
  
    int year;  
  
    // Parameterized constructor  
  
    Car(String b, int y) {  
  
        brand = b;  
  
        year = y;  
  
    }  
  
    // Copy constructor  
  
    Car(Car c) {  
  
        brand = c.brand;  
  
        year = c.year;  
  
    }  
  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car c1 = new Car("Tesla", 2023); // Original object  
  
        Car c2 = new Car(c1); // Copy object  
  
        System.out.println("Original Car: " + c1.brand + " " + c1.year);  
  
        System.out.println("Copied Car: " + c2.brand + " " + c2.year);  
  
    }  
  
}
```

```
}
```

How It Works

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

1. ``Car c1 = new Car("Tesla", 2023);`` → Creates the first object.
2. ``Car c2 = new Car(c1);`` → Calls the copy constructor, copying ``brand`` and ``year`` from ``c1``.
3. Both ``c1`` and ``c2`` have the same values, but they are different objects in memory.

2.4 CONSTRUCTOR OVERLOADING

Constructor overloading in Java is a feature allowing a single class to possess multiple constructors, each distinguished by a unique parameter list. This enables objects of that class to be initialized in various ways, depending on the specific data available or required at the time of object creation.

Key characteristics of constructor overloading:

Same Name, Different Parameters:

All overloaded constructors within a class share the same name (which is the class name), but their parameter lists must differ. This difference can be in:

Number of parameters: For example, one constructor takes no arguments, while another takes one argument. Multiple constructors in the same class with different parameter lists.

You can have multiple constructors with different parameter lists:

Example

```
class Car {  
  
    String brand;  
  
    int year;  
  
    Car() {  
  
        brand = "Unknown";  

```

```
year = 2000;
```

```
}
```

```
Car(String b) {
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
brand = b;
```

```
year = 2000;
```

```
}
```

```
Car(String b, int y) {
```

```
brand = b;
```

```
year = y;
```

```
}
```

```
}
```

```
public class Main {
```

```
public static void main(String[] args) {
```

```
Car c1 = new Car();
```

```
Car c2 = new Car("Honda");
```

```
Car c3 = new Car("Tesla", 2023);
```

```
System.out.println(c1.brand + " " + c1.year);
```

```
System.out.println(c2.brand + " " + c2.year);
```

```
System.out.println(c3.brand + " " + c3.year);
```

```
}
```

```
}
```

If your class contains mutable objects (like arrays, lists, or other objects), you may need a deep copy instead of a shallow copy.

The copy constructor is useful when:

- * You want to duplicate objects.

- * You want to avoid changes in one object affecting another.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

2.5 STATIC VARIABLES AND METHODS

In Java, the static keyword is used to declare members (variables and methods) that belong to the class itself, rather than to any specific instance (object) of that class.

Static Variables (Class Variables):

- **Belong to the class:**

There is only one copy of a static variable, shared by all instances of the class. •

Memory allocation:

Static variables are allocated memory only once, when the class is loaded into the Java Virtual Machine (JVM).

- **Access:**

They can be accessed directly using the class name (e.g., `ClassName.variableName`), without needing to create an object of the class.

- **Use cases:**

Commonly used for constants, counters, or data that needs to be shared and consistent across all objects of a class.

Static Methods (Class Methods):

- **Belong to the class:**

Static methods are associated with the class, not with any particular object. •

Access:

They can be called directly using the class name (e.g., `ClassName.methodName()`), without needing to create an object.

Static Method in Java With Examples

In Java, the static keyword is used to create methods that belongs to the class rather than

any specific instance of the class. Any method that uses the static keyword is referred to as a static method.

Features of Static Method:

- A static method in Java is associated with the class, not with any object or instance.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

- It can be accessed by all instances of the class, but it does not rely on any specific instance.
- Static methods can access static variables directly without the need for an object.
- They cannot access non-static variables (instance) or methods directly.
- Static methods can be accessed directly in both static and non-static contexts.

Syntax to Declare a Static Method

```
access_modifier static return_type methodName() {  
  
    // method body  
  
}
```

The name of the class can be used to invoke or access static methods.

Syntax to Call a Static Method

```
ClassName.methodName();
```

Example 1: Static Method Cannot Access Instance Variables

The JVM executes the static method first, even before creating class objects. So, static methods cannot access instance variables, as no object exists at that point.

```
// Java program to demonstrate that  
  
// the static method does not have  
  
// access to the instance variable  
  
import java.io.*;  
  
public class sclass {  
  
    // static variable
```

```
static int a = 40;
```

```
// instance variable
```

```
int b = 50;
```

```
void simpleDisplay()
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
{
```

```
System.out.println(a);
```

```
System.out.println(b);
```

```
}
```

```
// Declaration of a static method
```

```
static void staticDisplay()
```

```
{
```

```
System.out.println(a);
```

```
}
```

```
// main method
```

```
public static void main(String[] args)
```

```
{
```

```
sclass obj = new sclass();
```

```
obj.simpleDisplay();
```

```
// Calling static method
```

```
staticDisplay();
```

```
}
```

```
}
```

Output

40

50

40

Example 2: Static Methods Accessed from Both Static and Non-Static Methods

// Java program to demonstrate that

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

// in both static and non-static methods,

// static methods are directly accessed

```
import java.io.*;
```

```
public class sclass {
```

```
    static int num = 100;
```

```
    static String str = "Hello World";
```

```
    // This is Static method
```

```
    static void display()
```

```
{
```

```
    System.out.println("Static number is: " + num);
```

```
    System.out.println("Static string is: " + str);
```

```
}
```

```
    // non-static method
```

```
    void nonstatic()
```

```
{
```

```
    // our static method can accessed
```

```
    // in non static method
```

```
    display();
```

```

}

// main method

public static void main(String args[])

{

    sclass obj = new sclass();

    // This is object to call non static method
UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

    obj.nonstatic();

    // static method can called

    // directly without an object

    display();

}

}

```

Output

Static number is: 100

Static string is: Hello World

Static number is: 100

Static string is: Hello World

2.6 ACCESS SPECIFIERS IN JAVA

In Java, **access modifiers** are essential tools that define how the members of a class, like **variables**, **methods**, and even the **class** itself, can be accessed from other parts of our program. They are an important part of building secure and modular code when designing large applications. In this article, we will explore each modifier with examples to demonstrate its impact on Java development.

Types of Access Modifiers

There are 4 types of access modifiers available in Java:

1. Default - No keyword required
2. Private
3. Protected
4. Public

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Algorithm to Use Access Modifier in Java

Here's a basic algorithm for using access modifiers in Java:

1. Define a class: Create a class to represent the object you want to manage. **2. Define instance variables:** Inside the class, define variables for the data you want to manage. **3. Set an access modifier:**

- Use private for variables only accessible within the class.
- Use protected for variables accessible within the class and its subclasses. •

Use public for variables accessible from anywhere.

4. Use getter and setter methods: To access or modify variables, use getter (accessor) and setter (mutator) methods, even for public variables, to maintain encapsulation.

1. Default Access Modifier

When no access modifier is specified for a class, method, or data member, it is said to have the default access modifier by default. This means only classes within the same package can access it.

Example 1: Demonstrating Default Access Modifier Within the Same Package In this example, we will create two packages and the classes in the packages will be having the default access modifiers and we will try to access a class from one package from a class of the second package.

Sampleclass1.java

// default access modifier

package p1;

class Sampleclass1

{

void display()

{

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

System.out.println("Hello World!");

}

}

Sampleclass2.java:

package p2;

import p1.*; *// importing package p1*

// This class is having default access modifier

class Sampleclass2 {

public static void main(String args[]) {

// Accessing class Sampleclass1 from package p1

Sampleclass1 o = **new** new Sampleclass1();

o.display();

}

}

Explanation: In this example, the program will show the compile-time error when we try to access a default modifier class from a different package.

2. Private Access Modifier

The private access modifier is specified using the keyword private. The methods or data members declared as private are accessible only within the class in which they are declared. • Any other class of the same package will not be able to access these members. • Top-level classes or interfaces cannot be declared as private because, private means "only

visible within the enclosing class". Protected means "only visible within the enclosing class and any subclasses".

These modifiers in terms of application to classes, apply only to nested classes and not on top level classes.

Example: In this example, we will create two classes A and B within the same package p1. We will declare a method in class A as private and try to access this method from class B and see the result.

// Error while using class from different package with private access modifier

```
package p1;
```

```
class A {
```

```
    private void display() {
```

```
        System.out.println("Welcome");
```

```
UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA
```

```
    }
```

```
}
```

```
class B {
```

```
    public static void main(String args[]) {
```

```
        A obj = new A();
```

```
        // Trying to access private method of another class
```

```
        obj.display();
```

```
    }
```

```
}
```

Explanation: The above code will show a compile-time error when trying to access a private method from class B, even within the same package.

3. Protected Access Modifier

The protected access modifier is specified using the keyword protected. The methods or data members declared as protected are accessible within the same package or subclasses in different packages.

Example 1: In this example, we will create two packages p1 and p2. Class A in p1 is made public, to access it in p2. The method display in class A is protected and class B is inherited from class A and this protected method is then accessed by creating an object of class B. *// protected access modifier*

```
package p1;
```

```
public class A {
```

```
    protected void display() {
```

```
        System.out.println("Welcome");
```

```
}  
}
```

So, it demonstrates that a protected method is accessible within the same package.

Example 2: In this example, we will create two packages, p1 and p2. Class A in p1 has a protected method display. Class B in p2 extends A and accesses the protected method through inheritance by creating an object of class B.

```
package p2;
```

```
import p1.*;
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
class B extends A {
```

```
    public static void main(String args[]) {
```

```
        B obj = new B();
```

```
        obj.display();
```

```
    }
```

```
}
```

Explanation: The above example demonstrates that a protected method is accessible in a subclass from a different package using inheritance.

4. Public Access Modifier

The public access modifier is specified using the keyword public.

- The public access modifier has the widest scope among all other access modifiers.
- Classes, methods, or data members that are declared as public are accessible from everywhere in the program. There is no restriction on the scope of public data members.

Example 1: Here, the code shows that a public method is accessible within the same package. **package p1;**

```
public class A {
```

```
    public void display() {
```

```
        System.out.println("Welcome");
```

```
    }
```

```
}
```

Example 2: Here, the example shows that a public method is accessible across packages.

```
package p2;
```

```
import p1.*;
```



```

class B {
    public static void main(String args[]) {
        A obj = new A();
        obj.display();
    }
}

```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

2.7 METHOD OVERLOADING

In Java, Method Overloading allows us to define multiple methods with the same name but different parameters within a class. This difference may include:

- The number of parameters
- The types of parameters
- The order of parameters

Method overloading in Java is also known as Compile-time Polymorphism, Static Polymorphism, or Early binding, because the decision about which method to call is made at compile time. When there are overloaded methods that accept both a parent type and a child type, and the provided argument could match either one, Java prefers the method that takes the more specific (child) type. This is because it offers a better match.

Key features of Method Overloading

- Multiple methods can share the same name in a class when their parameter lists are different.
 - Overloading is a way to increase flexibility and improve the readability of code.
- Overloading does not depend on the return type of the method, two methods cannot be overloaded by just changing the return type.

Example: Program to find the area of square, rectangle, and circle by using method overloading.

```

import java.util.Scanner;

public class AreaCalculator {
    double area(double side) {
        return side * side;
    }

    double area(double length, double breadth) {

```

```

return length * breadth;
}
double area(float radius) {
return Math.PI * radius * radius;
}

```

```

public static void main(String[] args) {

```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```

Scanner sc = new Scanner(System.in);
AreaCalculator calc = new AreaCalculator();
System.out.print("Enter the side of the square: ");
double side = sc.nextDouble();
System.out.println("Area of Square: " + calc.area(side));
System.out.print("\nEnter the length of the rectangle: ");
double length = sc.nextDouble();
System.out.print("Enter the breadth of the rectangle: ");
double breadth = sc.nextDouble();
System.out.println("Area of Rectangle: " + calc.area(length, breadth));
System.out.print("\nEnter the radius of the circle: ");
float radius = sc.nextFloat();
System.out.println("Area of Circle: " + calc.area(radius));
sc.close();
}
}

```

Different Ways of Method Overloading in Java

The different ways of method overloading in Java are mentioned below:

- Changing the Number of Parameters
- Changing Data Types of the Arguments
- Changing the Order of the Parameters of Methods

1. Changing the Number of Parameters

Method overloading can be achieved by changing the number of parameters while passing to different methods.

Example: This example demonstrates method overloading by changing the number of parameters in the multiply() method.

Program

```
import java.io.*;
class Product {
    public int multiply(int a, int b)
    {
        UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```

```
        int prod = a * b;
        return prod;
    }
    public int multiply(int a, int b, int c)
    {
        int prod = a * b * c;
        return prod;
    }
}
class class1 {
    public static void main(String[] args)
    {
        Product ob = new Product();
        int prod1 = ob.multiply(1, 2);
        System.out.println(
            "Product of the two integer value: " + prod1);
        int prod2 = ob.multiply(1, 2, 3);
        System.out.println(
            "Product of the three integer value: " + prod2);
    }
}
```

Output

Product of the two integer value: 2

Product of the three integer value: 6

2. Changing Data Types of the Arguments

In many cases, methods can be considered overloaded if they have the same name but have different parameter types, methods are considered to be overloaded.

Example: This example demonstrates method overloading by changing the data types of parameters in the Prod() method.

Program

```
import java.io.*;
class Product {
UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA
```

```
    public int Prod(int a, int b, int c)
    {
        int prod1 = a * b * c;
        return prod1;
    }
    public double Prod(double a, double b, double c)
    {
        double prod2 = a * b * c;
        return prod2;
    }
}
class class1 {
    public static void main(String[] args)
    {
        Product obj = new Product();
        int prod1 = obj.Prod(1, 2, 3);
        System.out.println("Product of the three integer value: " + prod1);
        double prod2 = obj.Prod(1.0, 2.0, 3.0);
        System.out.println(
            "Product of the three double value: " + prod2);
    }
}
```

Output

Product of the three integer value: 6

Product of the three double value: 6.0

3. Changing the Order of the Parameters of Methods

Method overloading can also be implemented by rearranging the parameters of two or more overloaded methods. For example, if the parameters of method 1 are (String name, int roll_no) and the other method is (int roll_no, String name) but both have the same name, then these 2 methods are considered to be overloaded with different sequences of parameters.

Example: This example demonstrates method overloading by changing the order of parameters in the StudentId() method.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

Program

```
import java.io.*;

class Student {

    public void StudentId(String name, int roll_no)
    {
        System.out.println("Name: " + name + " " + "Roll-No: " + roll_no);
    }

    public void StudentId(int roll_no, String name)
    {
        // Again printing name and id of person
        System.out.println("Roll-No: " + roll_no + " " + "Name: " + name);
    }
}

class class1
{
    public static void main(String[] args)
    {
        Student obj = new Student();
        obj.StudentId("Sweta", 1);
        obj.StudentId(2, "Gudly");
    }
}
```

Output

Name: Sweta Roll-No: 1

Roll-No: 2 Name: Gudly

2.8 APPLICATION EXAMPLES

1. Student Record system

Consider a scenario of a school where everyday teachers, staff, authorities need to go through the records of their students for various purposes like searching for a particular student's details. Manually going through records is a tedious job and also time-consuming. Hence, it is

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

better to develop in-school software that allows users to insert, update, search, or delete records without manually going through documents every time a query arises.

We will see how to quickly create an application using Java Swing to perform operations like create, retrieve, and delete into the database using JDBC.

Before writing the code, a few things need to be kept in mind:

1. **JDBC API:** Java Database Connectivity Application Program Interface is a set of interfaces and classes using which you can write Java programs for accessing and manipulating databases. It acts as a communication between the application and the database.
2. **JDBC Driver:** It enables a Java application to interact with the database. We need to set up different JDBC drivers for different databases.



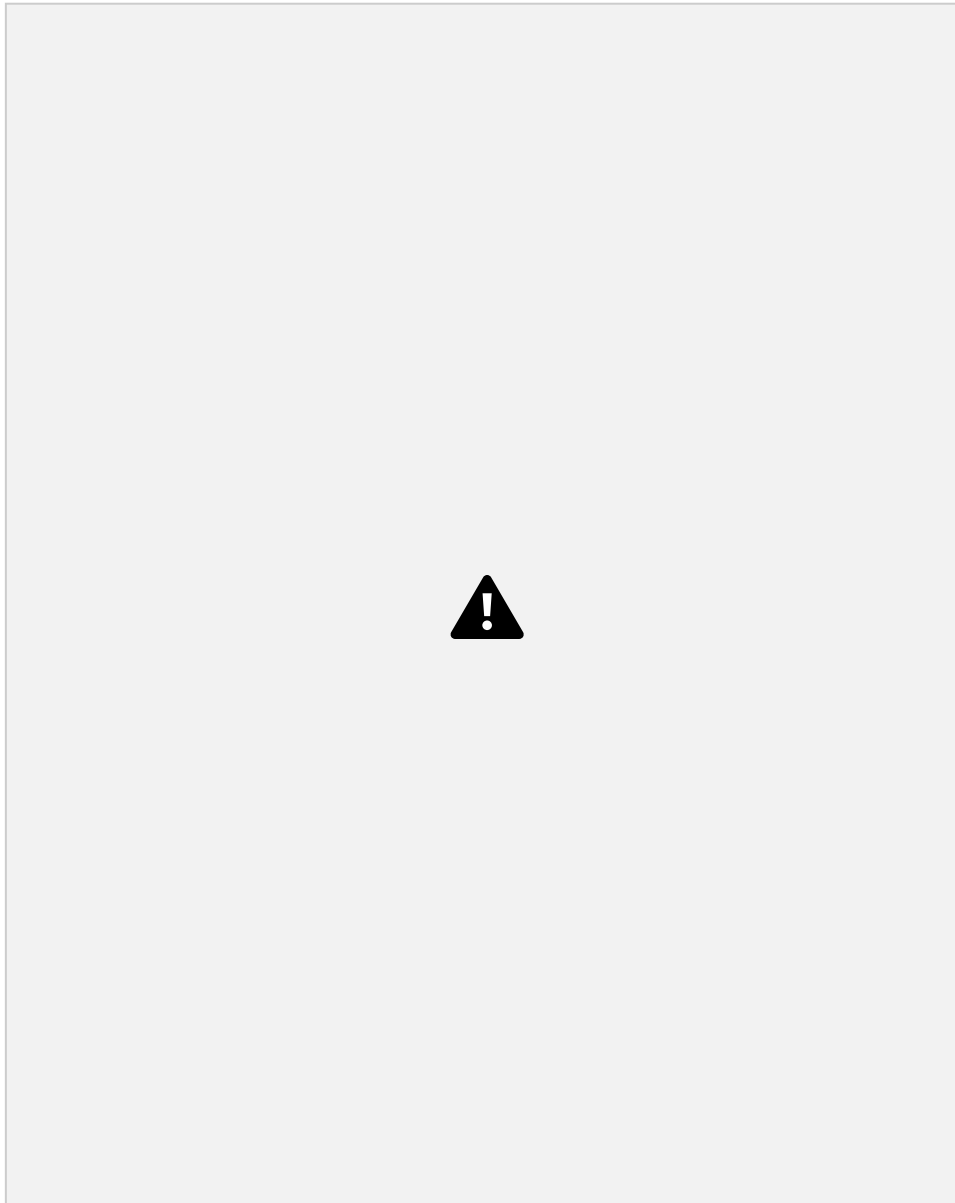
Workflow of a Java application and database interaction through JDBC drivers **Steps to create the application:**

1. First, open Netbeans and click on the File option from the menu bar.



UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

2. Now create a new Java application by clicking on **New Project -> Java -> Java Application** and give a suitable project name and click finish.



3. Now create a new file by going to the File option again on the menu bar, then **New File -> Swing GUI Forms -> JFrame Form**, and give a suitable file name click finish.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



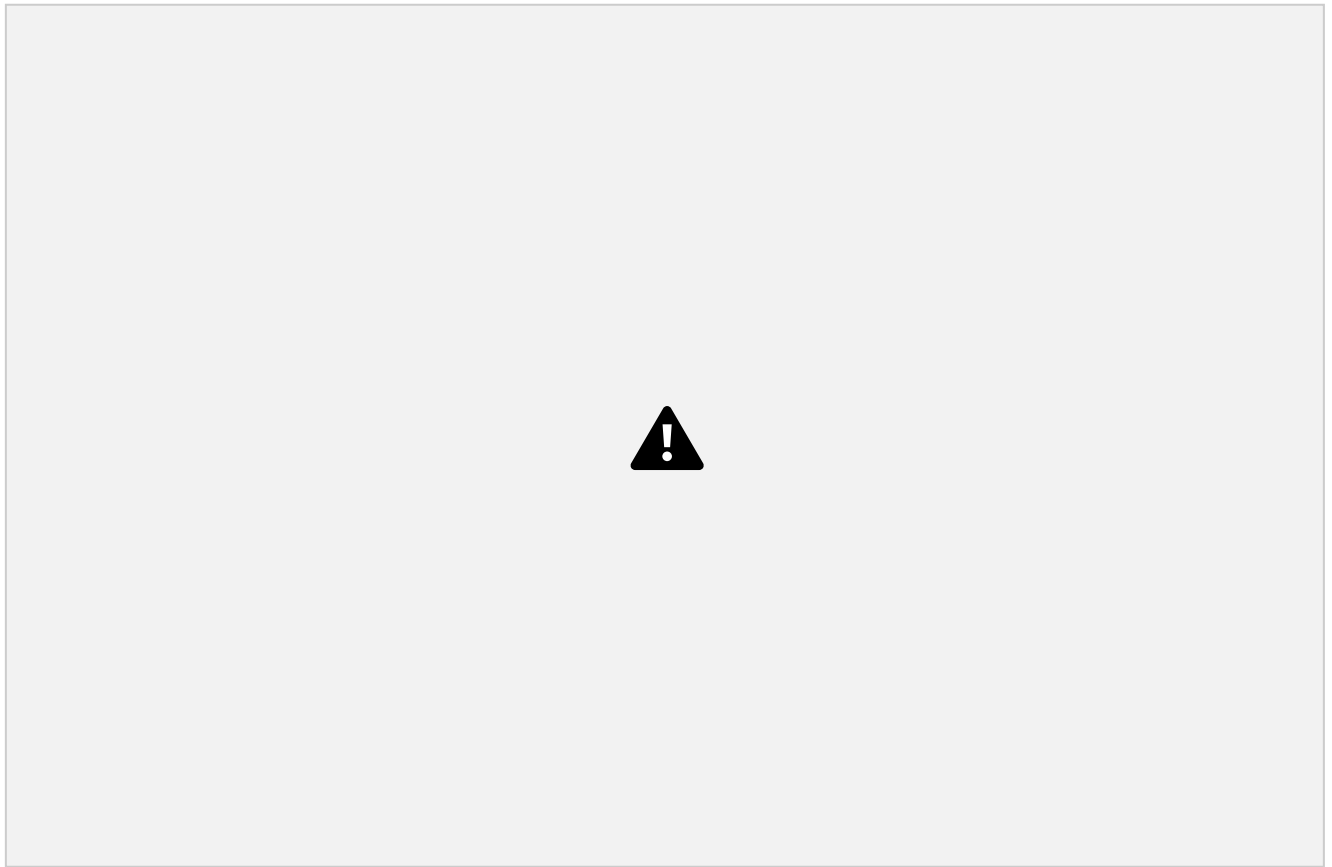
4. After successful file creation, we will now be presented with the following screen. The 3 important parts of this window are:

- **Design:** This is the area where we will create the design/template of our application. •

Source: This is where the logic code of the program is written.

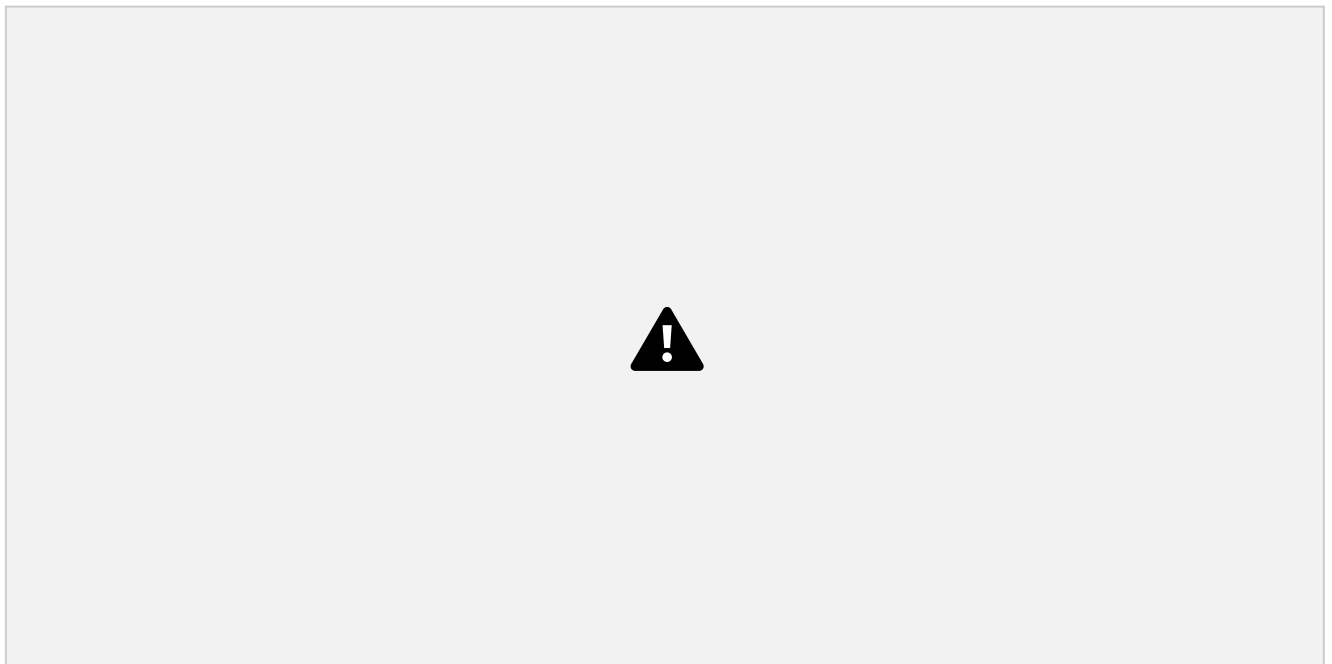
- **Palette:** This component contains all the widgets which we need to drag and drop on the design area

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Window displayed after successful Java file creation.

5. Now from the palette situated at the right-hand side of the window, start dragging the toolkit widgets.



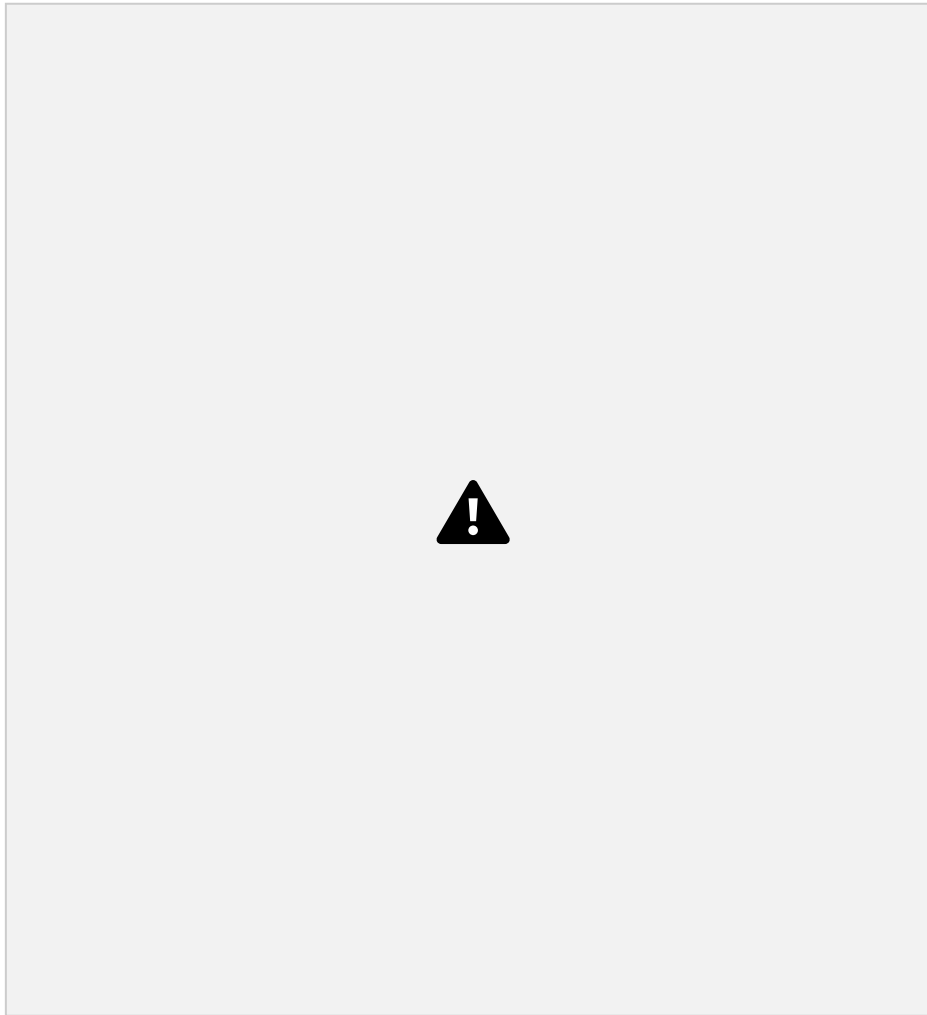
Drag Components from palette to design area

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

6. Since we need to display all data in a tabulated form, drag the table widget from the palette

onto the design area. Now to set the headings, right-click on the table, select **Properties** -> **Model** -> **Add/delete Columns**.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



7. Now let us create the database to store the data. Open MySQL command client, enter password, and type in the following commands to create a new database, new table, and defining the attributes.

```
mysql> create database student;
```

Query OK, 1 row affected (0.14 sec)

```
mysql> use student;
```

Database changed

```
mysql> create table record(
```

```
-> rollno int(3),
```

```
-> name char(20),
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

-> class int(2),

-> section char(2),

-> address varchar(40));

Query OK, 0 rows affected (2.03 sec)

mysql> describe record;

Field	Type	Null	Key	Default	Extra
rollno	int(3)	YES		NULL	
name	char(20)	YES		NULL	
class	int(2)	YES		NULL	
section	char(2)	YES		NULL	
address	varchar(40)	YES		NULL	

5 rows in set (0.03 sec)

Going back to NetBeans, we need to follow these steps for database connectivity:

8. We need to import libraries that are needed to set up a connection with the database and retrieve data which is done by - *DriverManager Class*, *Connection Class*, and *Statement Class*. Thus go to the menubar, under **Tools->Libraries**, and add *MySQL JDBC connector*. Note down the Library ClassPath and click OK. Now go to **Projects** toolbar and go to your application's Libraries. Right-click and select **Add Jar/Library** and browse the Library classpath noted down previously.

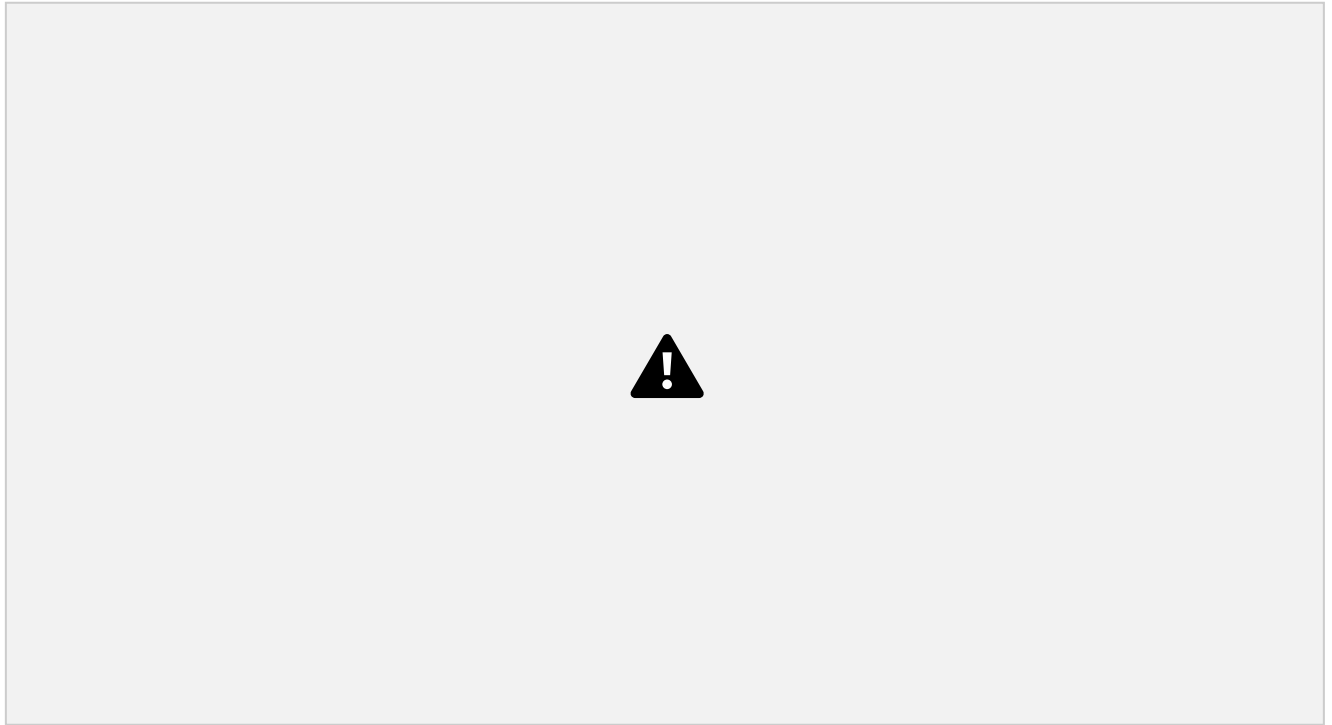
UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



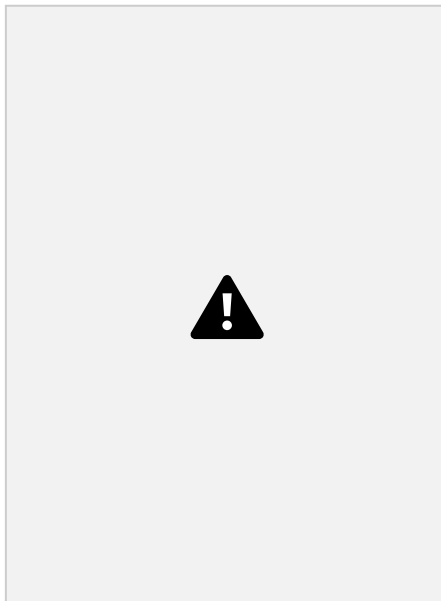
Add Jar file



Add mysql connector

9. Go to **Windows->Services->Databases** and enter the required credentials of your MySQL username and password. After clicking test connection, if it's successful, the connector logo appears connected.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Connection is initially broken

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



10. Now to type the code, double-click on jButton1 (Insert), you will be directed to the source tab. Here type in the following code.

```
// Write the import code at top
import java.sql.*;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
public class SchoolRecord extends javax.swing.JFrame {
    Connection con = null;
    Statement stmt = null;
    ResultSet rs = null;
    // JButton1 Code starts from here
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```

try {
    Class.forName("com.mysql.jdbc.Driver");
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/student", "root",
        "root");
    stmt = con.createStatement();
    String rollno = jTextField1.getText();
    String name = jTextField2.getText();
    String class = jTextField3.getText();
    String sec = jTextField4.getText();
    String adr = jTextArea1.getText();
    String INSERT = "INSERT INTO RECORD VALUES("
        + rollno + "," + name + ","
        + class + "," + sec + "," + adr
        + ")";
    stmt.executeUpdate(INSERT);
    JOptionPane.showMessageDialog(
        this, "Record Added Successfully");
    jButton1.setEnabled(true);
}
catch (Exception e) {
    JOptionPane.showMessageDialog(
        this, "Error In Connectivity");
}

```

In the above code, the following things need to be kept in mind which are:

1. **Connection Class:** It acts as a connection session between the Java program and specific database application. It is through which we send SQL queries to the database.
2. **Statement Class:** A Statement is an interface that represents a SQL statement.
3. **ResultSet:** When you execute Statement objects, and they generate ResultSet objects, which is a table of data representing a database result set. A Connection object is needed to create a Statement object.

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

4. **JDBC Driver Registration:** To open a connection to the database from Java application, the JDBC driver should be registered with the Device Manager. Hence we use **forName()** of language package. Here *com.mysql.jdbc.Driver* is the driver name for MySQL.
5. **.getConnection():** It is used to establish a physical connection to the database by specifying the *database name(student)* , *username(root)* and *password(root)*. This creates a connection object.
6. **Query Execution:** *createStatement()* creates a statement type object that holds SQL queries. Then *executeQuery/executeUpdate* method executes the SQL statement. Here it is "INSERT INTO RECORD VALUES....".
7. **Data Extraction:** The above method creates a resultset object that contains the resultant data. (*Now see the code below*) *rs* is the variable that stores the resultant dataset and hence we use a *.get<Type>()* method to obtain data.
8. **while(rs.next()):** (*See code below*) Since we need data containing multiple rows, we use a loop to access them. The *next()* method moves the cursor forward by one row.
9. **Closing open databases:** We close all open databases to clean up the environment. Thus we use- *rs.close()* , *stmt.close()* and *con.close()* methods.
11. Now to display all data, write the following code under the jButton2 ActionPerformed option which can be achieved by clicking twice on *View Data Button* in the design area.

DefaultTableModel model

```
= (DefaultTableModel) jTable1.getModel();  
  
try {  
    Class.forName("com.mysql.jdbc.Driver");  
    Connection con = DriverManager.getConnection(  
        "jdbc:mysql://localhost:3306/student", "root",  
        "root");  
    String query = "SELECT* FROM RECORD;";  
    ResultSet rs = stmt.executeQuery(query);  
    while (rs.next()) {  
        String rollno = rs.getString("rollno");  
        String name = rs.getString("name");  
        String class = rs.getString("class");  
        String sec = rs.getString("section");
```

```
String adr = rs.getString("address");
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
model.addRow(  
    new Object[] { rollno, name, class, sec, adr });  
}  
rs.close();  
stmt.close();  
con.close();  
}  
catch (Exception e) {  
    JOptionPane.showMessageDialog(this,  
    "Error In Connectivity");  
}
```

12. Now to clear all textfields, textareas and table content write the following code under the jButton3 ActionPerformed option which can be achieved by clicking twice on Clear Button in the design area.

```
jTextField1.setText("");  
jTextField2.setText("");  
jTextField3.setText("");  
jTextField4.setText("");  
jTextArea1.setText("");  
DefaultTableModel dm  
= (DefaultTableModel)jTable1.getModel();  
dm.getDataVector().removeAllElements();  
jTable1.repaint();
```

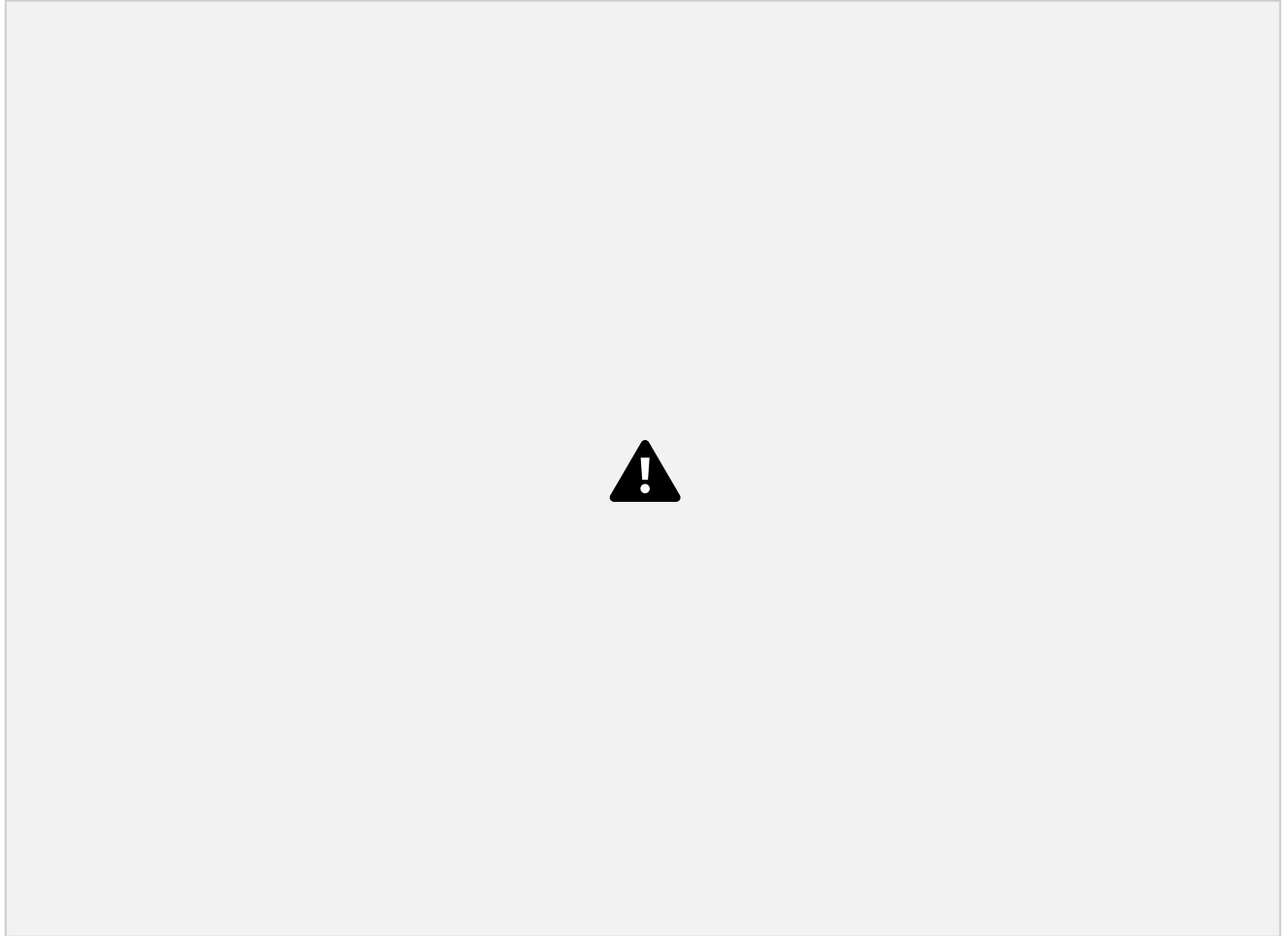
13. Now to exit from the system, add the following statement under the jButton4 ActionPerformed option which can be achieved by clicking twice on Exit Button in the design area.

```
System.exit(0);
```

14. After code is typed, right-click anywhere on the screen and select the Run File option from

the drop-down menu. The final output is shown below. Input necessary details and the application is ready!

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



Insert Data

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA



2. BASIC USER REGISTRATION MODELS

Swing is a part of the **JFC (Java Foundation Classes)**. Building Graphical User Interface in Java requires the use of Swings. **Swing Framework** contains a large set of components which allow a high level of customization and provide rich functionalities, and is used to create window-based applications. Java swing components are lightweight, platform independent, provide powerful components like tables, scroll panels, buttons, list, color chooser, etc.

In this article, we'll see **how to make a Registration form** which includes all the buttons and field in one Form.

Steps:

1. Create a Java file that contains the main class - Registration. This class will only contain the main method to invoke the required methods.

```
class Registration {
```

```
public static void main(String[] args)
```

```
throws Exception
```

```
{
```

```
MyFrame f = new MyFrame();
```

```
}
```

```
}
```

2. Create another class MyFrame, which will contain the form.

3. In this MyFrame Class, the methods to be made are:

- Components like JLabel, TextField, JRadioButton, ButtonGroup, JComboBox, and JTextArea.

These components will collectively form the Registration form. • A constructor, to initialize the components with default values.

- A method **actionPerformed()** to get the action performed by the user and act accordingly. • 4.

Copy the code of MyFrame class from below.

- 5. Save the file as Registration.java

6. Compile the file by using javac command.

```
javac Registration.java
```

7. Run the program by calling the main class

```
java Registration
```

Below is the code to implement the Simple Registration Form using Java Swing:

```
// Java program to implement
```

```
// a Simple Registration Form
```

```
// using Java Swing
```

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
class MyFrame
```

```
extends JFrame
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
implements ActionListener {  
    // Components of the Form  
    private Container c;  
    private JLabel title;  
    private JLabel name;  
    private JTextField tname;  
    private JLabel mno;  
    private JTextField tmno;  
    private JLabel gender;  
    private JRadioButton male;  
    private JRadioButton female;  
    private ButtonGroup gengp;  
    private JLabel dob;  
    private JComboBox date;  
    private JComboBox month;  
    private JComboBox year;  
    private JLabel add;  
    private JTextArea tadd;  
    private JCheckBox term;  
    private JButton sub;  
    private JButton reset;  
    private JTextArea tout;  
    private JLabel res;  
    private JTextArea resadd;  
    private String dates[]  
    = { "1", "2", "3", "4", "5",  
        "6", "7", "8", "9", "10",  
        "11", "12", "13", "14", "15",  
        "16", "17", "18", "19", "20",  
        "21", "22", "23", "24", "25",  
        "26", "27", "28", "29", "30",  
        "31" };  
    private String months[]
```


UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
= { "Jan", "feb", "Mar", "Apr",  
    "May", "Jun", "July", "Aug",  
    "Sep", "Oct", "Nov", "Dec" };  
private String years[]  
= { "1995", "1996", "1997", "1998",  
    "1999", "2000", "2001", "2002",  
    "2003", "2004", "2005", "2006",  
    "2007", "2008", "2009", "2010",  
    "2011", "2012", "2013", "2014",  
    "2015", "2016", "2017", "2018",  
    "2019" };  
  
// constructor, to initialize the components  
// with default values.  
public MyFrame()  
{  
    setTitle("Registration Form");  
    setBounds(300, 90, 900, 600);  
    setDefaultCloseOperation(EXIT_ON_CLOSE);  
    setResizable(false);  
    c = getContentPane();  
    c.setLayout(null);  
    title = new JLabel("Registration Form");  
    title.setFont(new Font("Arial", Font.PLAIN, 30));  
    title.setSize(300, 30);  
    title.setLocation(300, 30);  
    c.add(title);  
  
    name = new JLabel("Name");  
    name.setFont(new Font("Arial", Font.PLAIN, 20));  
    name.setSize(100, 20);  
    name.setLocation(100, 100);  
    c.add(name);
```

```
tname = new JTextField();
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
tname.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
tname.setSize(190, 20);
```

```
tname.setLocation(200, 100);
```

```
c.add(tname);
```

```
mno = new JLabel("Mobile");
```

```
mno.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
mno.setSize(100, 20);
```

```
mno.setLocation(100, 150);
```

```
c.add(mno);
```

```
tmno = new JTextField();
```

```
tmno.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
tmno.setSize(150, 20);
```

```
tmno.setLocation(200, 150);
```

```
c.add(tmno);
```

```
gender = new JLabel("Gender");
```

```
gender.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
gender.setSize(100, 20);
```

```
gender.setLocation(100, 200);
```

```
c.add(gender);
```

```
male = new JRadioButton("Male");
```

```
male.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
male.setSelected(true);
```

```
male.setSize(75, 20);
```

```
male.setLocation(200, 200);
```

```
c.add(male);
```

```
female = new JRadioButton("Female");
```

```
female.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
female.setSelected(false);
```

```
female.setSize(80, 20);
```

```
female.setLocation(275, 200);
```

```
c.add(female);
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
gengp = new ButtonGroup();
```

```
gengp.add(male);
```

```
gengp.add(female);
```

```
dob = new JLabel("DOB");
```

```
dob.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
dob.setSize(100, 20);
```

```
dob.setLocation(100, 250);
```

```
c.add(dob);
```

```
date = new JComboBox(dates);
```

```
date.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
date.setSize(50, 20);
```

```
date.setLocation(200, 250);
```

```
c.add(date);
```

```
month = new JComboBox(months);
```

```
month.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
month.setSize(60, 20);
```

```
month.setLocation(250, 250);
```

```
c.add(month);
```

```
year = new JComboBox(years);
```

```
year.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
year.setSize(60, 20);
```

```
year.setLocation(320, 250);
```

```
c.add(year);
```

```
add = new JLabel("Address");
```

```
add.setFont(new Font("Arial", Font.PLAIN, 20));
```

```
add.setSize(100, 20);
```

```
add.setLocation(100, 300);
```

```
c.add(add);
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
tadd = new JTextArea();
```

```
tadd.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
tadd.setSize(200, 75);
```

```
tadd.setLocation(200, 300);
```

```
tadd.setLineWrap(true);
```

```
c.add(tadd);
```

```
term = new JCheckBox("Accept Terms And Conditions.");
```

```
term.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
term.setSize(250, 20);
```

```
term.setLocation(150, 400);
```

```
c.add(term);
```

```
sub = new JButton("Submit");
```

```
sub.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
sub.setSize(100, 20);
```

```
sub.setLocation(150, 450);
```

```
sub.addActionListener(this);
```

```
c.add(sub);
```

```
reset = new JButton("Reset");
```

```
reset.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
reset.setSize(100, 20);
```

```
reset.setLocation(270, 450);
```

```
reset.addActionListener(this);
```

```
c.add(reset);
```

```
tout = new JTextArea();
```

```
tout.setFont(new Font("Arial", Font.PLAIN, 15));
```

```
tout.setSize(300, 400);
tout.setLocation(500, 100);
tout.setLineWrap(true);
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
tout.setEditable(false);
c.add(tout);
```

```
res = new JLabel("");
res.setFont(new Font("Arial", Font.PLAIN, 20));
res.setSize(500, 25);
res.setLocation(100, 500);
c.add(res);
```

```
resadd = new JTextArea();
resadd.setFont(new Font("Arial", Font.PLAIN, 15));
resadd.setSize(200, 75);
resadd.setLocation(580, 175);
resadd.setLineWrap(true);
c.add(resadd);
```

```
setVisible(true);
}
```

```
// method actionPerformed()
```

```
// to get the action performed
```

```
// by the user and act accordingly
```

```
public void actionPerformed(ActionEvent e)
```

```
{
```

```
if (e.getSource() == sub) {
```

```
if (term.isSelected()) {
```

```
String data1;
```

```
String data
```

```
= "Name : "
```

```
+ tname.getText() + "\n"
+ "Mobile : "
+ tmno.getText() + "\n";
```

```
if (male.isSelected())
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
data1 = "Gender : Male"
```

```
+ "\n";
```

```
else
```

```
data1 = "Gender : Female"
```

```
+ "\n";
```

```
String data2
```

```
= "DOB : "
```

```
+ (String)date.getSelectedItemAt()
```

```
+ "/" + (String)month.getSelectedItemAt()
```

```
+ "/" + (String)year.getSelectedItemAt()
```

```
+ "\n";
```

```
String data3 = "Address : " + tadd.getText();
```

```
tout.setText(data + data1 + data2 + data3);
```

```
tout.setEditable(false);
```

```
res.setText("Registration Successfully..");
```

```
}
```

```
else {
```

```
tout.setText("");
```

```
resadd.setText("");
```

```
res.setText("Please accept the"
```

```
+ " terms & conditions..");
```

```
}
```

```
}
```

```
else if (e.getSource() == reset) {
```

```
String def = "";
```

```
tname.setText(def);
```

```
tadd.setText(def);
tmno.setText(def);
res.setText(def);
tout.setText(def);
term.setSelected(false);
```

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

```
date.setSelectedIndex(0);
month.setSelectedIndex(0);
year.setSelectedIndex(0);
resadd.setText(def);
}
}
}
```

// Driver Code

```
class Registration {

public static void main(String[] args) throws Exception
{
    MyFrame f = new MyFrame();
}
}
```

1. Compile:



UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

2. Registration Form unfilled:



3. Registration Form filled:

UNIT II 24CS302 - OBJECT ORIENTED PROGRAMMING USING JAVA

